```
            t1.Start();
            t2.Start();
             // The original thread needs to join the two
threads we create.  That
              // way this main thread does not end (and
terminate the program) until
            // t1 and t2 are done.
            t1.Join();
            t2.Join();

            Console.ReadLine();
        }
       // Value types can't be the subject of locking.  So
we have to
        // explicitly create a wrapper object.
        class IntWrapper
        {
            public int val = 0;
        }
        // Our static counter, shared by t1 and t2
        static IntWrapper counter = new IntWrapper();
        static void PrintSeq()
        {
            // t1 and t2 each have their own copy of i
            for (int i = 0; i < 100; i++)
            {
                lock(counter)
                {
                   Console.WriteLine(Thread.CurrentThread.
Name +
                                                 ": {0}, {1}", i,
counter.val++);
                }
            }
        }
    }
```

Conceptually a lock is a token that threads can acquire and release. If a thread tries to acquire a lock that another thread holds, then that thread